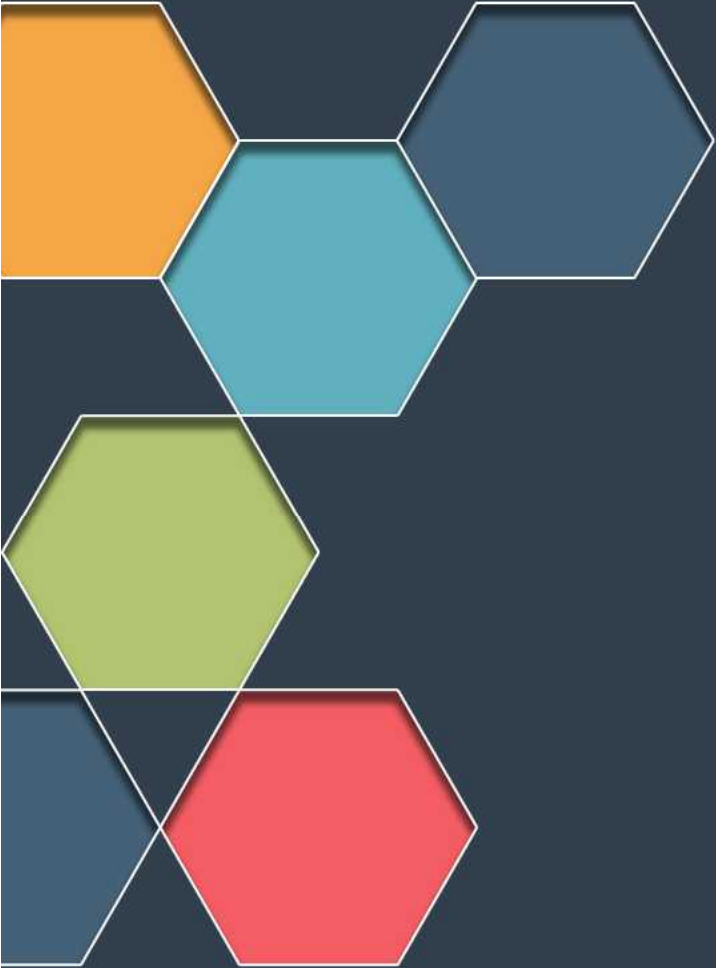




제15장 신경망Ⅳ (CNN: Convolutional Neural Network)

학습 목표

- 컨벌루션 신경망을 이해한다.
- 컨벌루션 신경망을 이용한 영상 인식(image recognition)을 이해한다.

컨볼루션 신경망 (Convolutional Neural Network: CNN)

노드가 컨볼루션이라는 노드를 사용한다

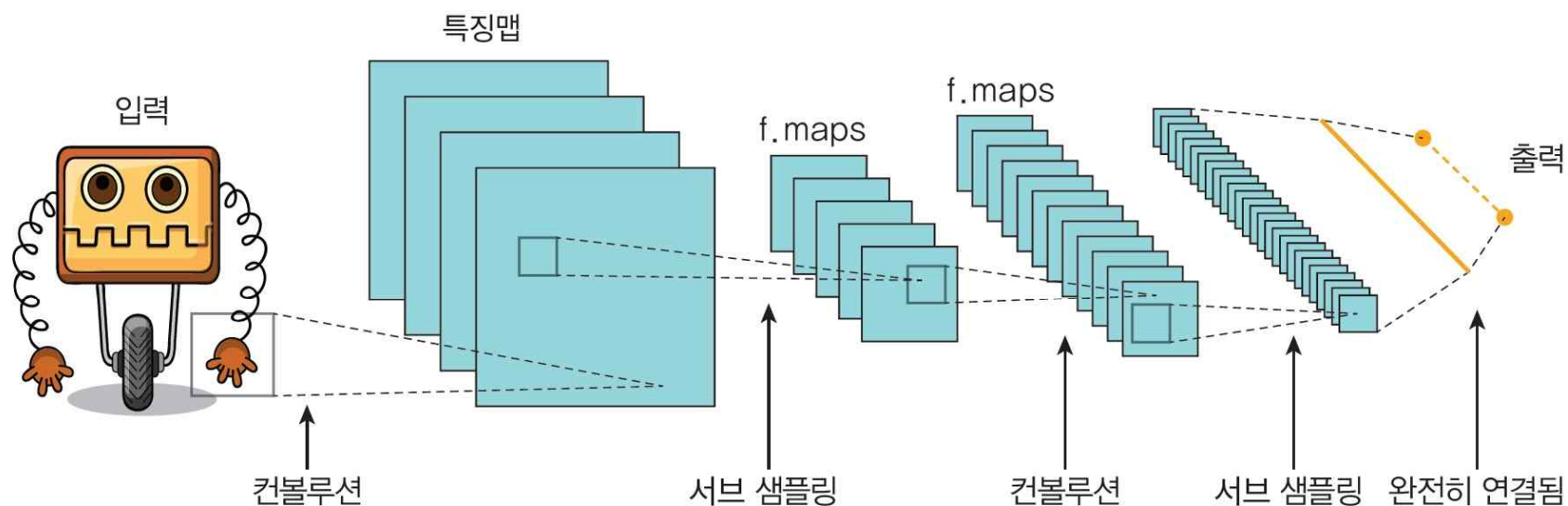
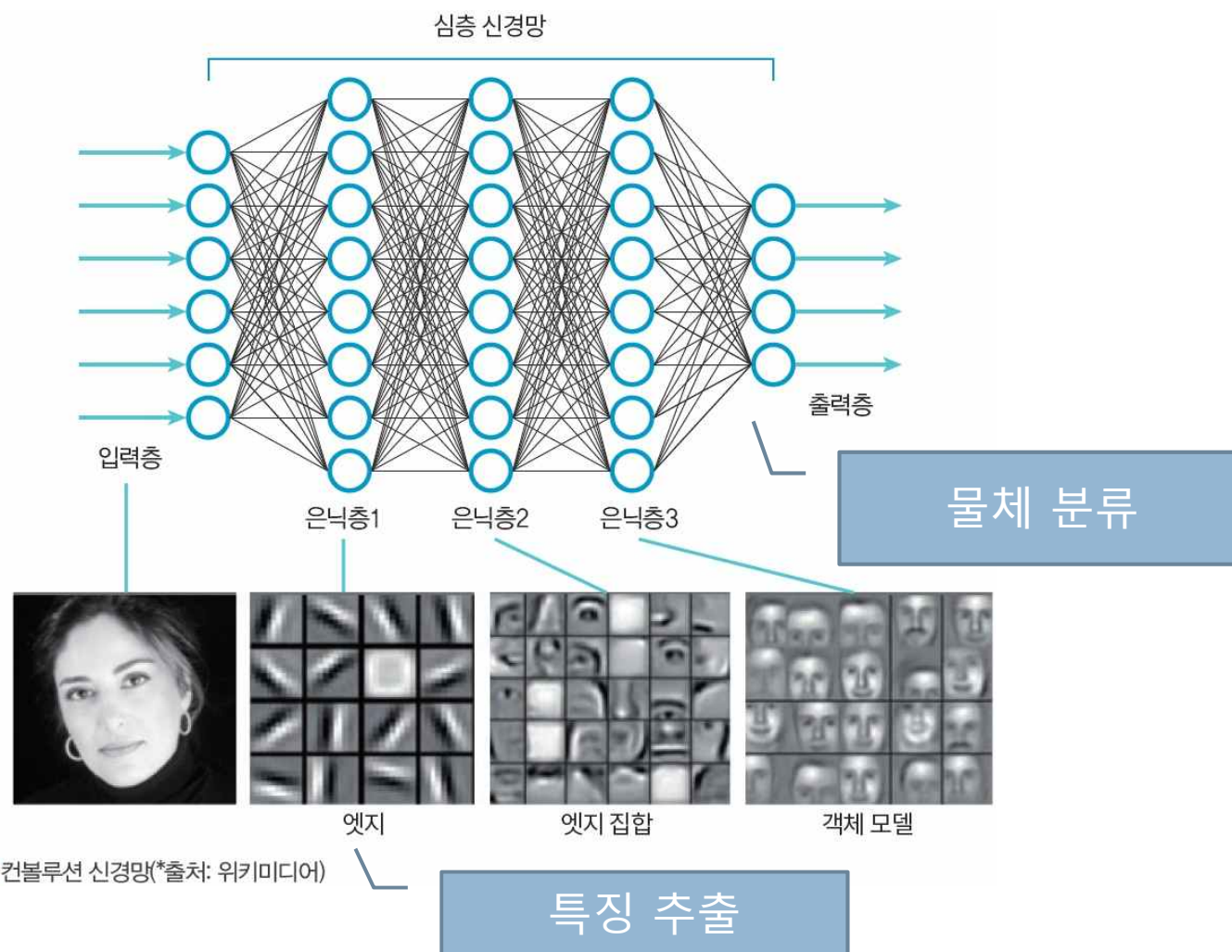


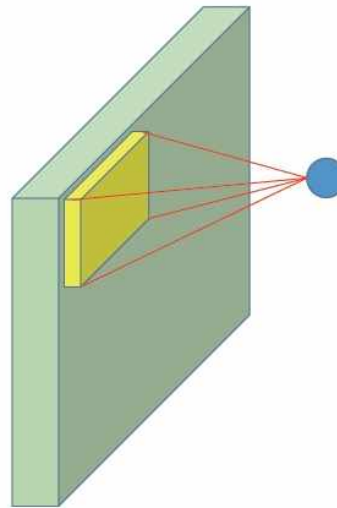
그림 15-10 컨볼루션 신경망의 구조(*출처: 위키미디어)

컨볼루션 신경망 (Convolutional Neural Network: CNN)

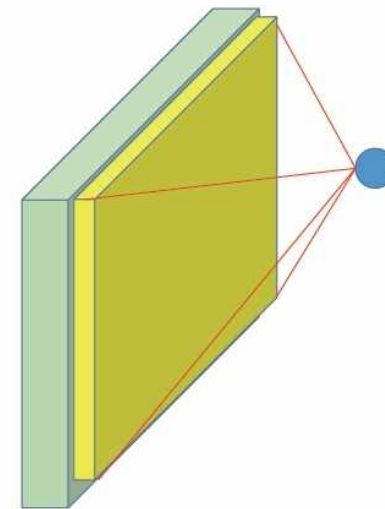


컨벌루션 신경망(CNN)

- 컨벌루션(Convolution Neural Network: CNN) 신경망에서는 하위 레이어의 노드들과 상위 레이어의 노드들이 부분적으로만 연결되어 있다.



(a) 컨볼루션 신경망



(b) 완전 연결 신경망

그림 15-11 컨볼루션 신경망과 완전 연결 신경망의 비교

컨벌루션 신경망(CNN)

- 컨벌루션 신경망의 유닛 사이의 연결 패턴은 고양이의 시각 세포에서 영감을 얻었다. 1962년 두 명의 노벨상 신경학자 허벨(Hubel)과 위젤(Wiesel)은 시각 피질에서 단순 세포와 복합 세포라고 하는 두 가지 유형의 세포를 발견했다

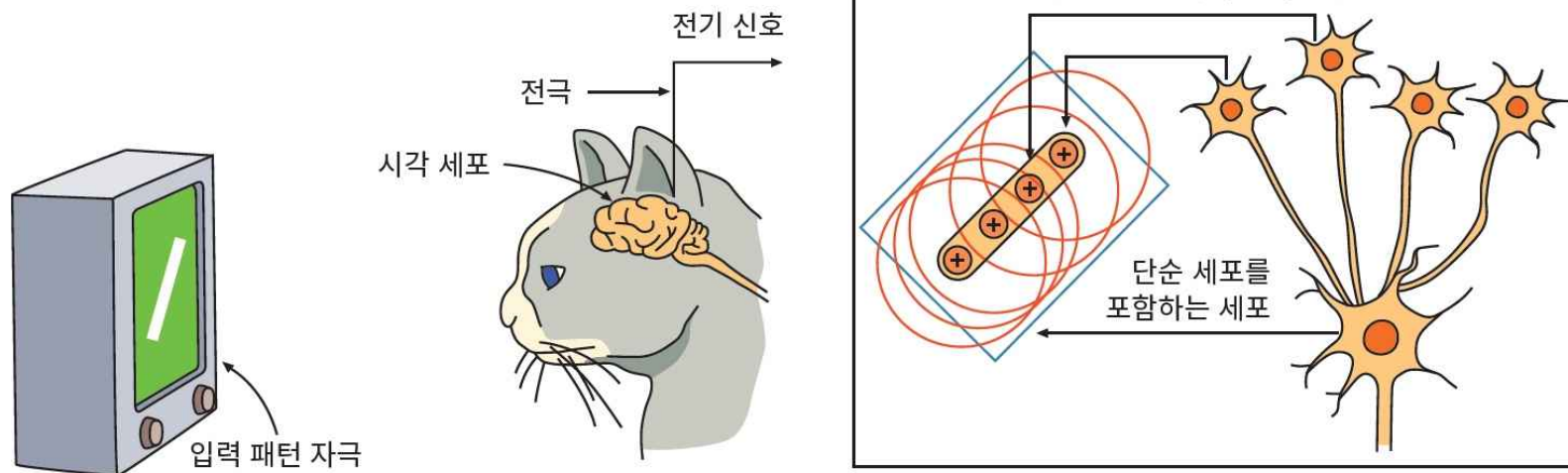


그림 15-12 수용장의 구조(그림 출처: Hubel's book, Eye, Brain, and Vision)

네오코그니트론

- ‘네오코그니트론(Neocognitron)’은 1980년 쿠니히코 후쿠시마(Kunihiko Fukushima)에 의해 소개된 신경망 구조이다.
- 후쿠시마도 앞에서 언급한 허벨과 위젤의 작업에서 영감을 받았다. 후쿠시마는 허벨과 위젤이 발견한 단순 세포와 복합 세포를 패턴 인식에 사용하기 위해, 이 두 가지 유형의 세포로 이루어진 **S층**(국소적인 엣지/방향 검출, 가중치 공유), **C층(pooling)**이 반복되는 계층적인 모델을 제안했다.
- 네오코그니트론에서는 컨볼루션 신경망의 두 가지 기본 유형 레이어인, 컨볼루션 레이어와 서브 샘플링 레이어를 도입했다.

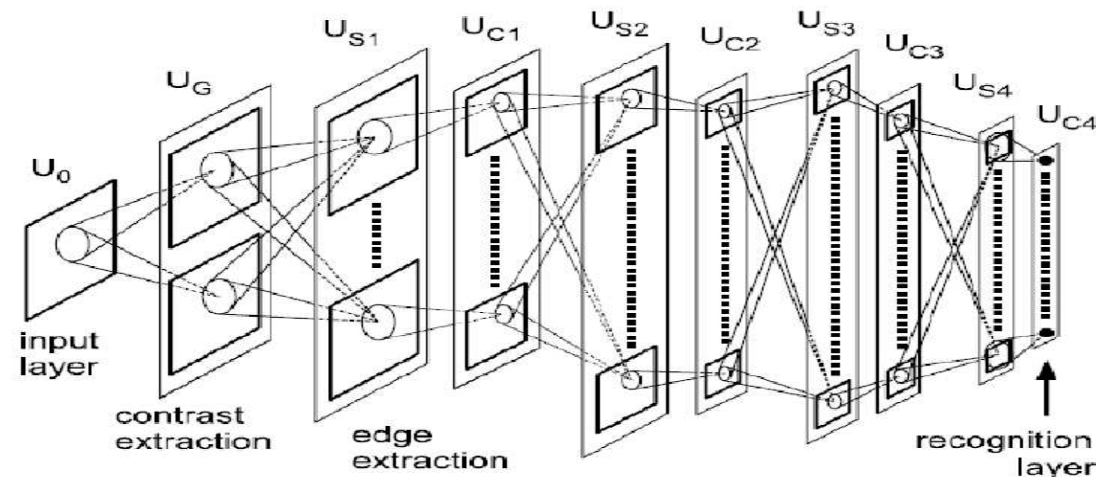


그림 15-3 네오코그니트론(출처: 후쿠시마의 논문)

컨볼루션 신경망의 중요성

- 컨볼루션 신경망은 모든 신경망 구조 중에서 가장 강력한 성능을 보여주는 신경망 중의 하나이다.
- 여러 가지 문제에 대해서도 좋은 성능을 보이고 있지만 가장 두드러진 성공은 영상 인식 분야이다.

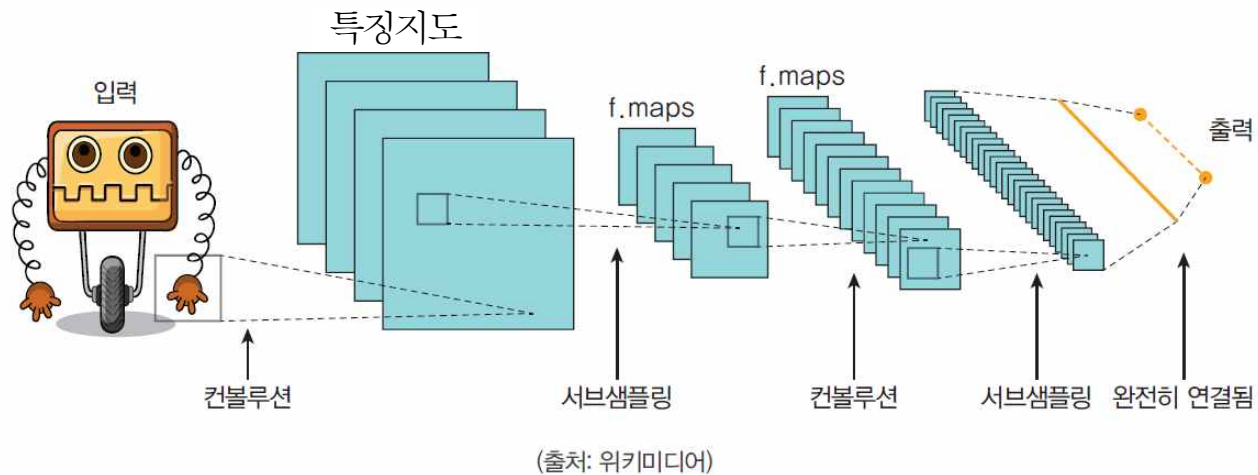


그림 15-4 영상 인식을 위한 컨볼루션 신경망의 구조

컨볼루션 신경망의 용도

- 컨볼루션 신경망은 이미지의 종류를 판별하거나 이미지 안의 물체를 추적하거나 숫자나 얼굴 인식, 이미지의 내용을 이해하여 이미지를 분류하거나 주석을 붙이는 작업에도 성공하고 있다.

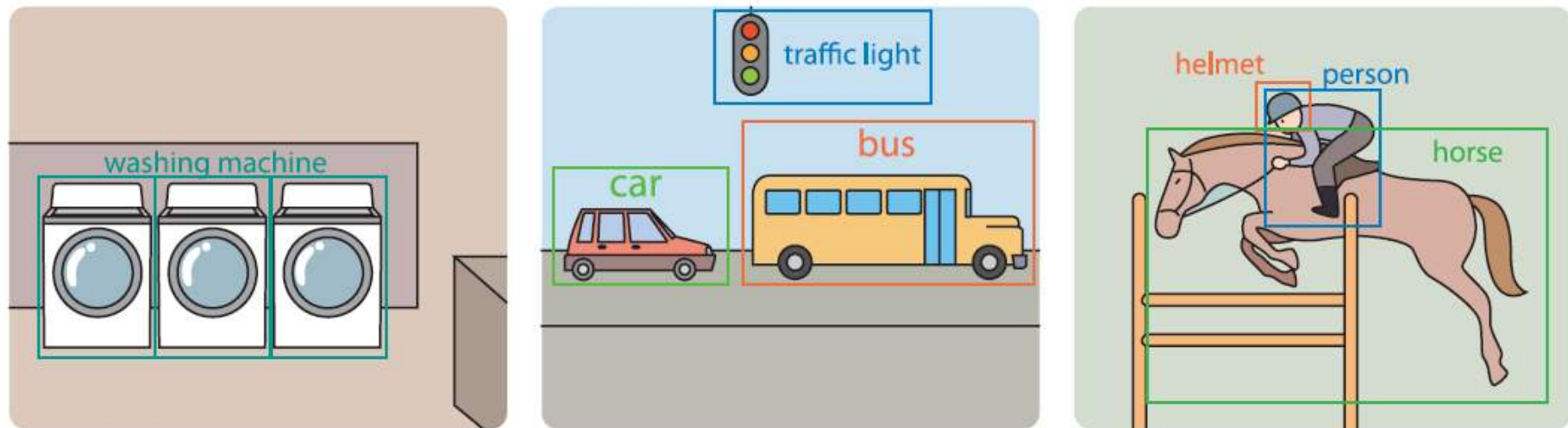


그림 15-5 컨볼루션 신경망은 주로 영상 인식에 많이 사용된다(출처 : Google research).

Lab: 컨벌루션 네트워크 체험하기

- <https://transcranial.github.io/keras-js/#/>

The screenshot displays the Keras.js web application interface. On the left, a sidebar lists various models under 'DEMOS' and 'LINKS'. The main area shows the 'Inception v3' model selected. A URL is entered in the 'enter image url' field, and a 'fox' image is selected from a dropdown. The 'use GPU' toggle is turned on. The central part of the interface shows a visualization of the fox image with 'Class Activation Mapping' and 'Transparency' options. To the right, the inference results are displayed, showing the top predicted classes and their probabilities. Below the image, a diagram of the model architecture is visible, starting with an 'InputLayer' and followed by 'Conv2D', 'BatchNormalization', and 'Activation' layers.

Keras.js

enter image url: https://farm4.staticflickr.com/3852/14447103450_2d0ff8802b... or select image: fox

use GPU: ☒

visualization: Class Activation Mapping

colormap: Transparency

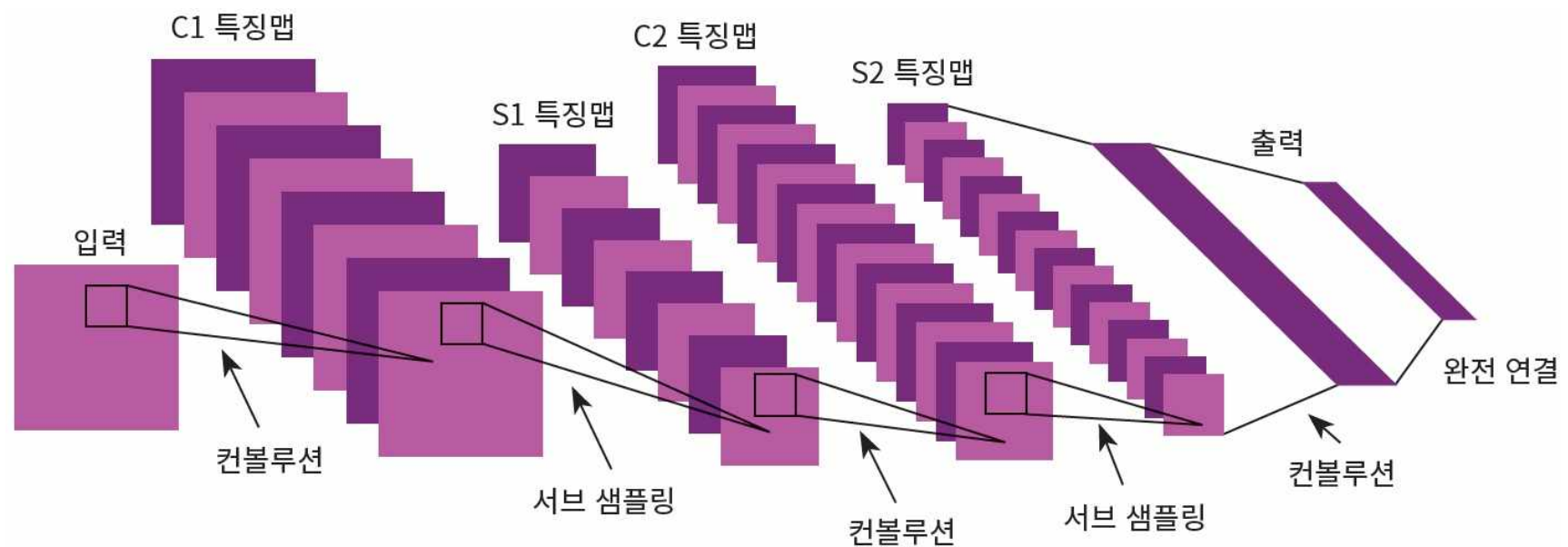
inference time: 551.6 ms (1.8 fps)

Class	Probability
red fox	67%
kit fox	13%
grey fox	6%
dhole	0%
lesser panda	0%

Model Architecture Diagram:

- InputLayer (shape: [299, 299, 3])
- Conv2D (32 3x3 filters, 2x2 striding, no border padding)
- BatchNormalization
- Activation (relu)

CNN의 일반적인 구조



CNN도 여러 층을 붙여서 신경망을 구축한다.

그림 15-13 CNN의 한 예(출처: 위키백과)

CNN의 일반적인 구조

- 입력층
- 입력층에서 컨벌루션 연산을 통하여 특징을 뽑아내는 특징맵(feature map)이 존재 – 가중치 학습
- 풀링(Pooling) 연산을 적용한다. 풀링 연산은 입력의 차원을 줄이는 연산이다. – 학습 아님
- 컨벌루션 레이어(가중치 학습)와 풀링 레이어(크기 축소)는 여러 번 되풀이 된다.
- 신경망의 맨 끝에는 완전히 연결된 구조의 전통적인 분류 신경망이 있어서 추출된 특징을 바탕으로 물체를 인식한다.

컨볼루션 연산

- 컨볼루션은 주변 화소값들에 가중치를 곱해서 더한 후에 이것을 새로운 화소값으로 하는 연산이다.

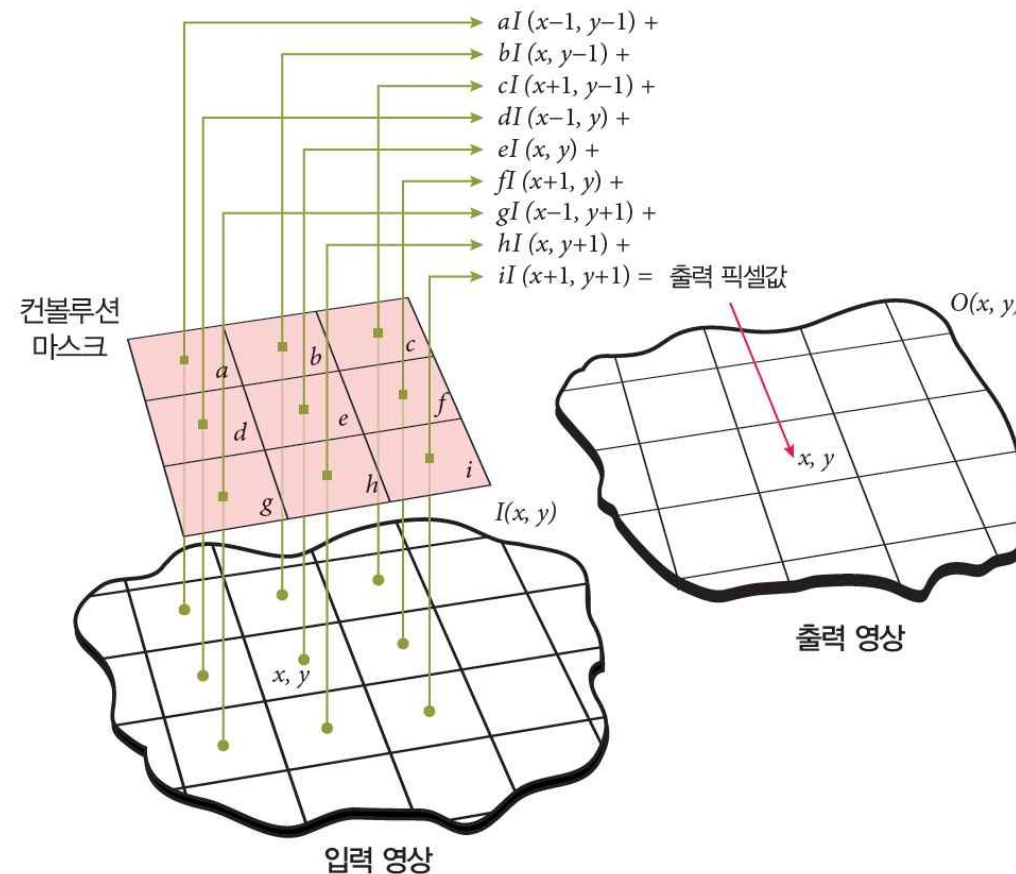


그림 15-14 컨볼루션의 설명

컨벌루션의 구체적인 예

3x3, 5x5

1대1로 다 대응을 해줘서 더한다.
가 합에다가 활성화 함수를 적용을 해준

$$\text{ReLU}(1/9 \times 3 + 1/9 \times 6 + 1/9 \times 6 + 1/9 \times 3 + 1/9 \times 4 + 1/9 \times 3 + 1/9 \times 5 + 1/9 \times 7 + 1/9 \times 7) = 4.88$$

활성화 함수를 통과해주면 0보다 크면 그대로 나오고, 0보다 작으면 0이 된다.
이렇게 처리하는 것이 컨벌루션 층의 역할이다.

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

가중치 마스크

3	6	6	4	7	8	2	1
3	4	3	8	8	3	3	2
5	7	7	7	7	4	3	2
8	9	9	9	9	9	3	2
8	3	3	4	3	2	1	1
8	9	9	8	8	3	3	2
6	4	3	8	8	3	3	2
7	4	3	8	8	3	3	2

입력 계층

	4.88						

다음 계층

그림 15-16 컨볼루션

컨벌루션의 구체적인 예

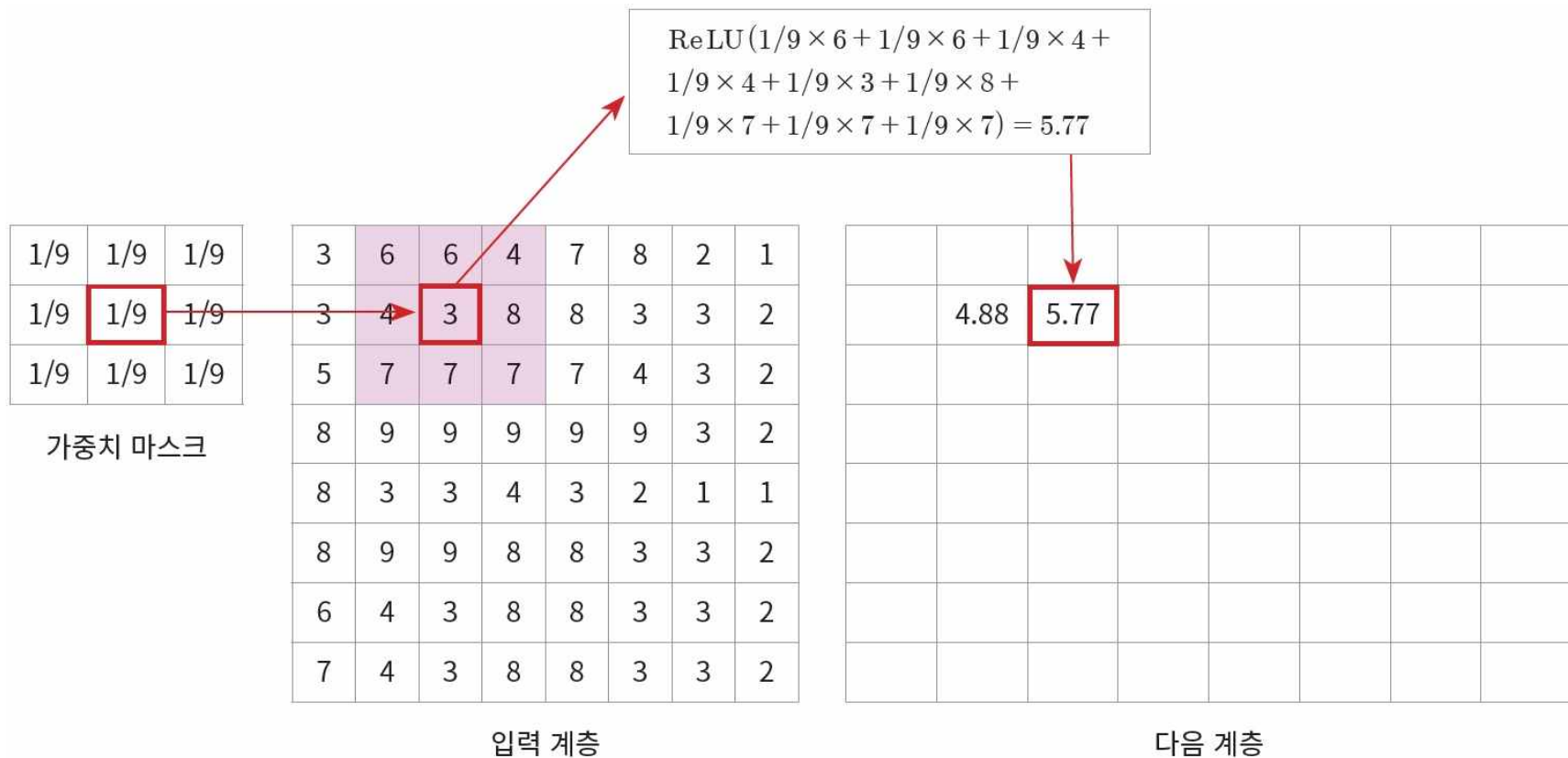


그림 15-17 컨볼루션

영상 처리에서 컨볼루션 연산 결과

- 구체적인 예로 아래 그림과 같이 가운데만 8이고 나머지는 -1인 필터를 가정하자. 이 필터는 라플라시안 연산자라고 불리며 영상에서 에지(edge)를 찾는 필터이다. 이 필터를 영상에 적용하면 그림 15-11의 오른쪽과 같은 결과가 얻어진다

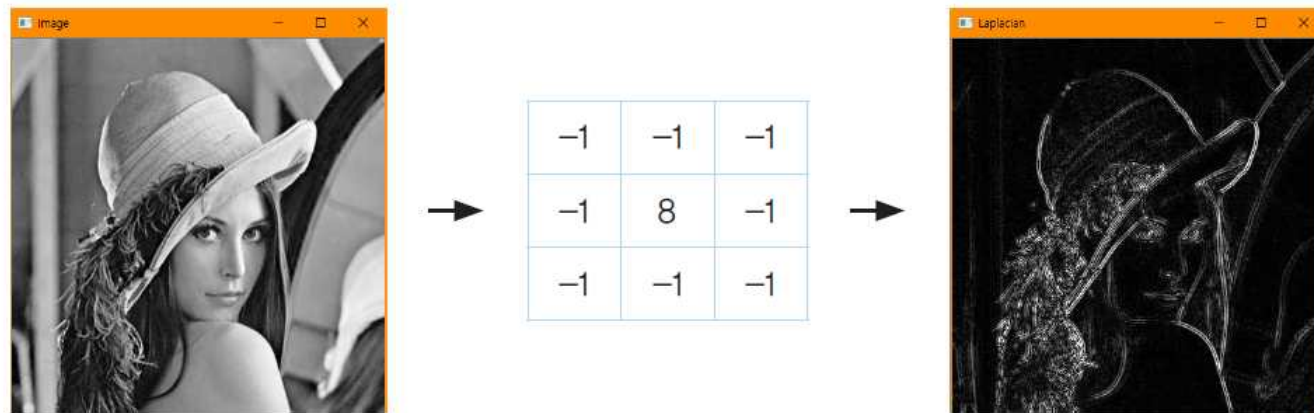


그림 15-11 영상처리에서의 컨볼루션 연산

신경망에서의 컨볼루션 연산

- CNN에서는 어떤 특징을 추출하려고 특정한 가중치를 가지는 필터를 미리 만드는 것이 아니다. 컨볼루션 신경망에서 어떤 컨볼루션 필터가 어떤 특징을 추출하는지는 우리도 알 수 없다.

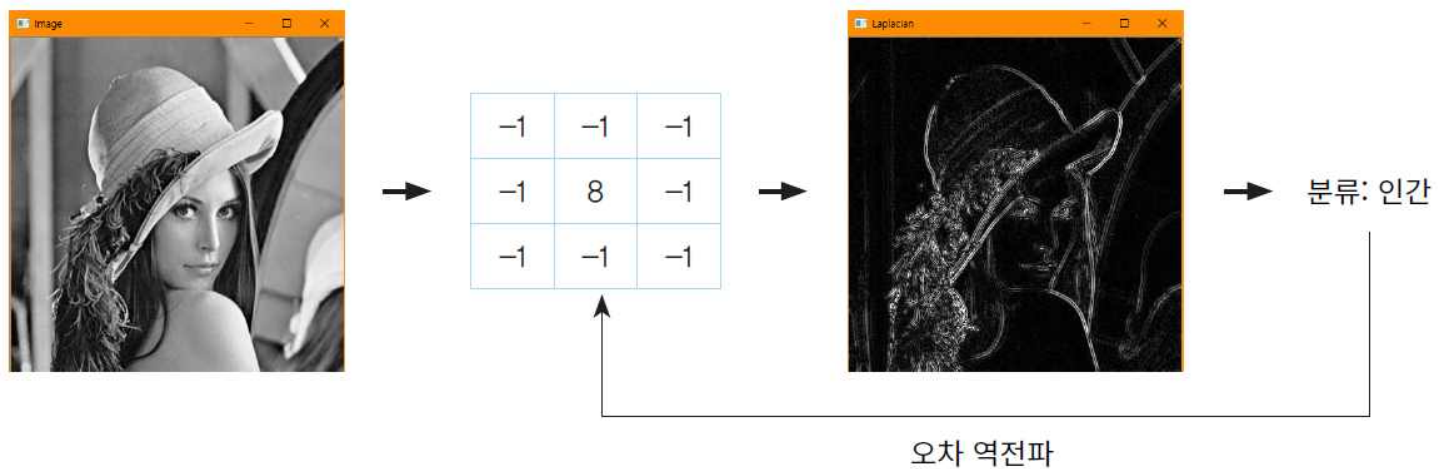


그림 15-12 컨볼루션 신경망에서는 커널의 가중치들이 학습된다.

컨볼루션 신경망에서의 컨볼루션 연산

- 앞 레이어의 값 X 는 각커널 W 와 곱해지고 더해져서 $WX+b$ 가 된다. 컨볼루션 신경망도 바이어스를 가진다. 이 계산값은 $\text{Relu}()$ 와 같은 활성화 함수를 통과해서, 다음 레이어의 동일한 위치에 $\text{ReLU}(WX+b)$ 로 저장된다. 이어서 가중치 필터는 한 칸 이동한다

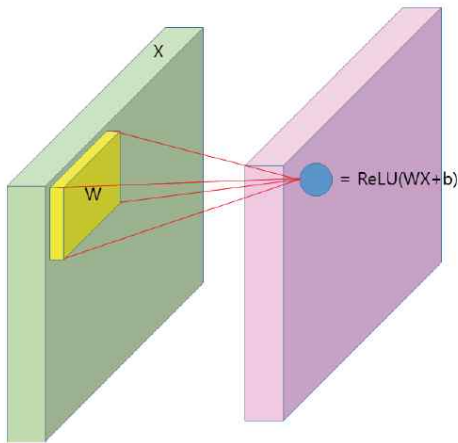


그림 15-13 신경망에서의 컨볼루션 연산

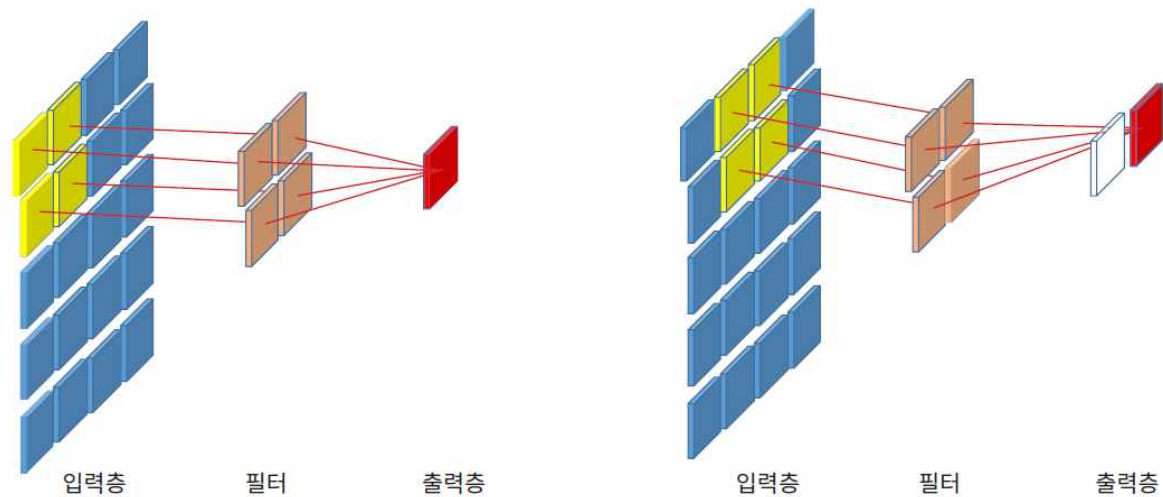


그림 15-15 컨볼루션 연산

Q

- 컨벌루션 연산에 대하여 설명하라.

보폭

- 보폭(stride)은 커널을 적용하는 거리이다. 보폭이 1이면 커널을 한 번에 1픽셀씩 이동하면서 커널을 적용하는 것이다. 보폭이 2라는 것은 2픽셀씩 건너뛰면서 픽셀에 커널을 적용한다는 것을 의미한다

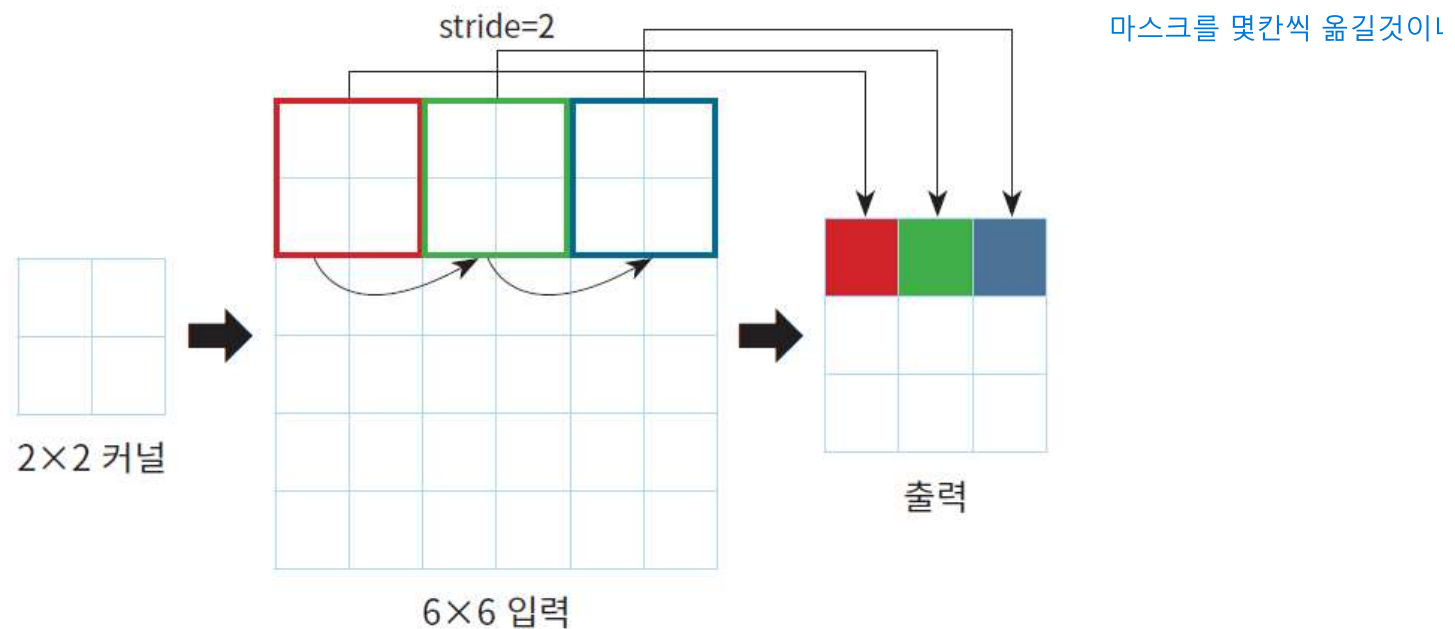
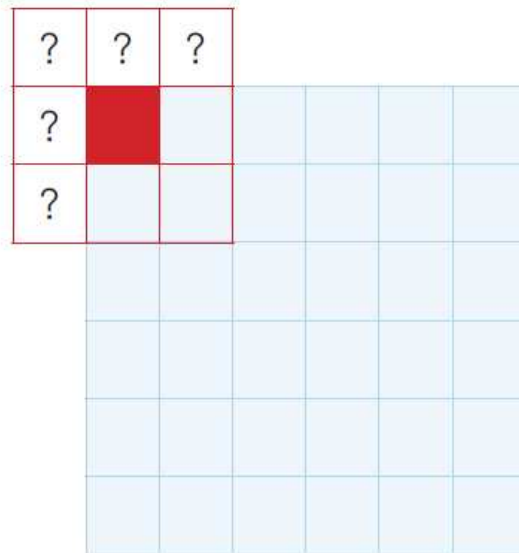


그림 15-16 컨볼루션 신경망에서의 보폭의 개념

패딩

- 패딩(padding)은 이미지의 가장자리를 처리하기 위한 기법이다. 아래 그림과 같이 가장자리 픽셀에 커널을 적용할 때는 어떻게 해야 할까?

가장자리에 있는 애들은 결과가 안나온다, 그래서 결과가 크기가 줄어들게 패딩을 안하는 경우 이런식으로 크기가 줄어든다.



이미지의 가장자리에 커널을 적용하려니, 커널 아래에 픽셀이 없네요. 어떻게 해야 할까요?



패딩을 하는 경우 가장자리가 안줄어든게 된다.

그림 15-17 패딩이 필요한 이유

패딩: valid (keras에서) – 덧대지 않고 줄어듦

$$\text{출력} = \text{입력} - (\text{kernel_size} - 1), (H - k + 1) \times (W - k + 1)$$

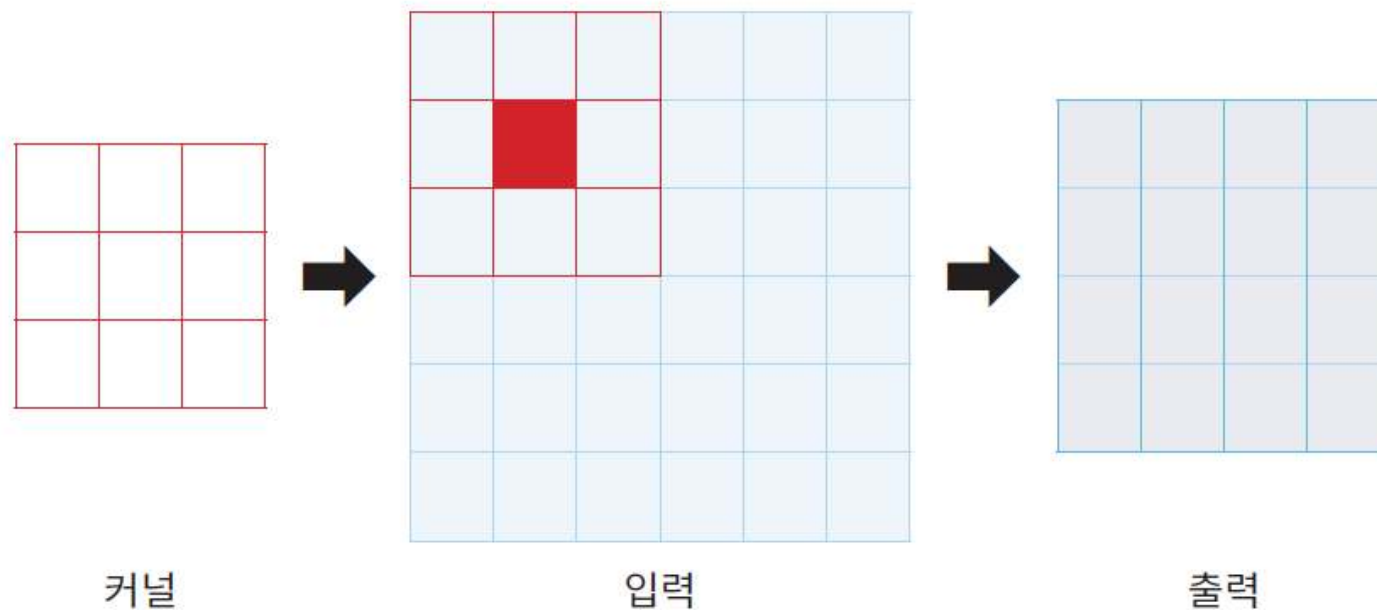


그림 15-18 패딩을 valid로 설정하는 경우

패딩: same (keras 에서)

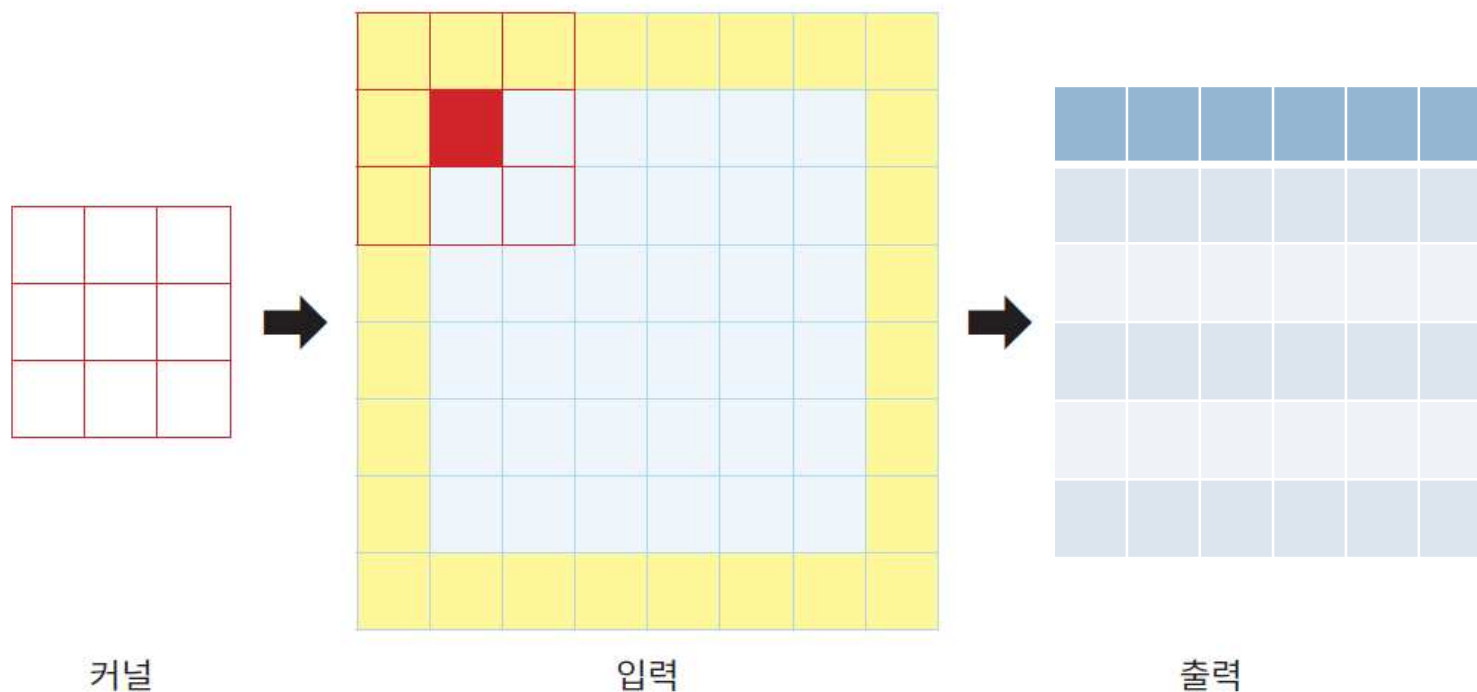


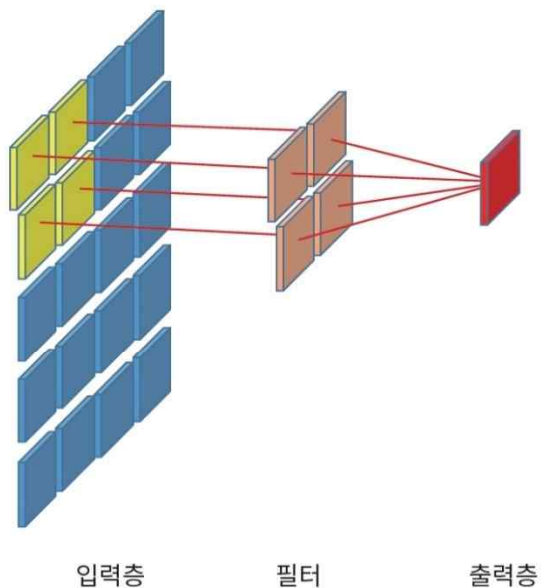
그림 15-19 패딩을 same으로 설정하는 경우

“padding='valid'는 출력 크기가 매 층마다 무조건 줄어듭니다. 그래서 2025년 실무에서는 거의 안 쓰고, same 패딩 또는 stride=2, MaxPooling으로 크기를 조절합니다!”

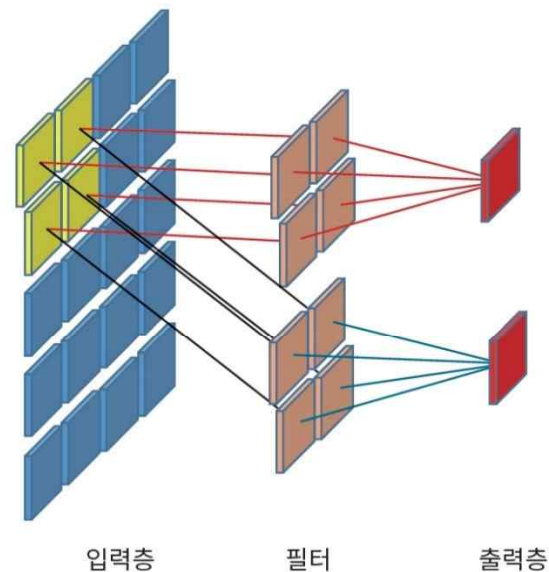
padding = valid 사용 안하겠다는 뜻
padding = same 패딩을 사용해서 크기 똑같이 하

컨볼루션 레이어

- 여러 개의 필터를 이용할 수 있다.
- *필터의 값은 미리 정해진 것이 아니고 학습된다.*



(a) 필터가 하나일 때



(b) 필터가 2개일 때

그림 15-23 컨볼루션 계층

커널의 개수

- 하나의 컨볼루션 레이어에서 여러 개의 커널을 동시에 학습하는 경우도 많다. 128개나 256개의 커널도 흔히 사용된다.

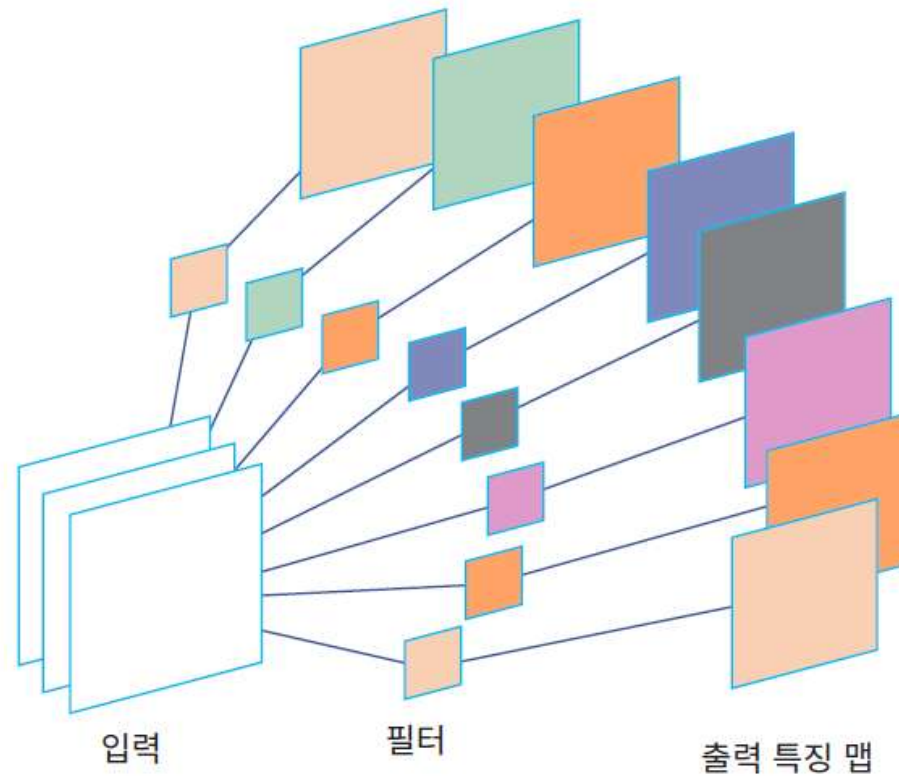
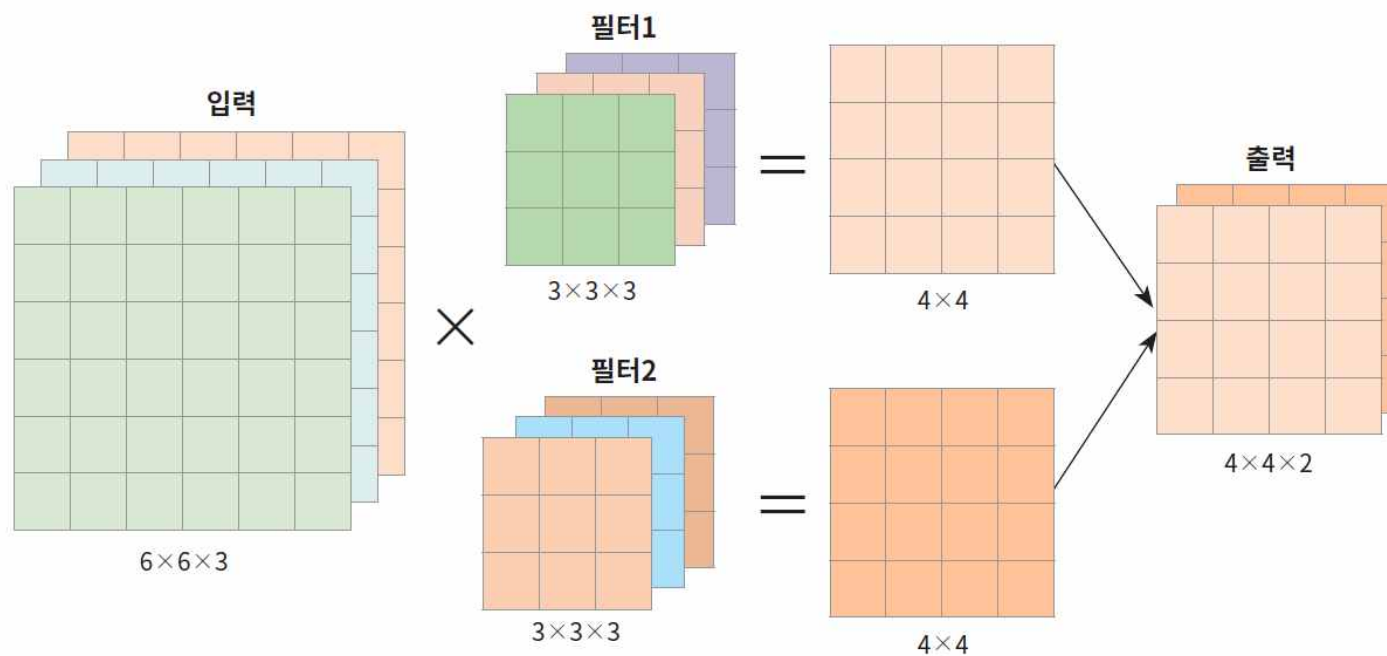


그림 14-20 필터가 여러 개일 때의 컨볼루션 레이어

여러 개의 커널을 적용하는 컨볼루션 레이어의 예



여러 개의 커널을 적용하는 컨볼루션 레이어의 예

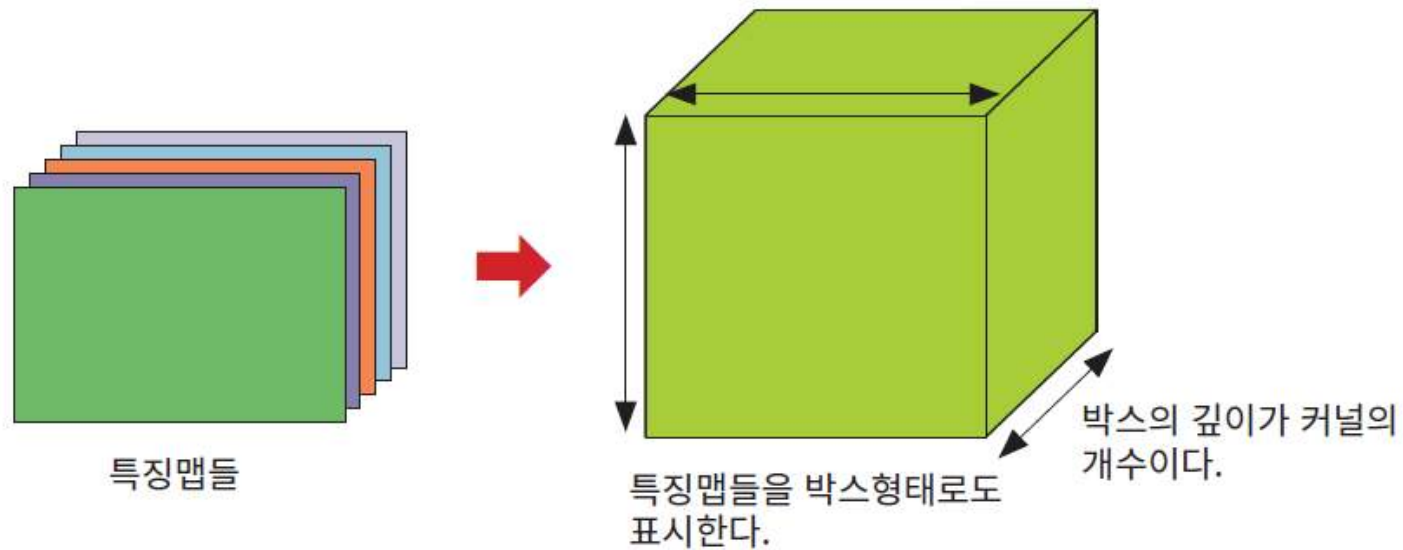
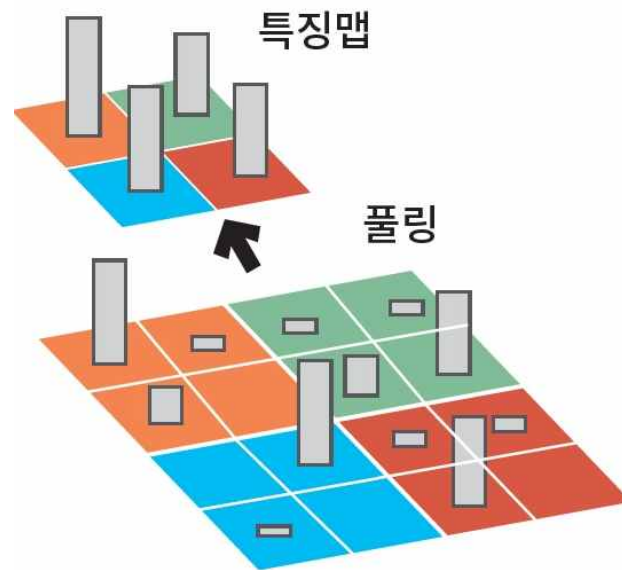


그림 15-22 컨볼루션 레이어를 박스 형태로 표시

풀링 또는 서브 샘플링

- 풀링(Pooling)이란 서브 샘플링이라고도 하는 것으로 **입력 데이터의 크기를 줄이는 것**이다.



풀링 또는 서브 샘플링

2x2

최대 값으로 풀링

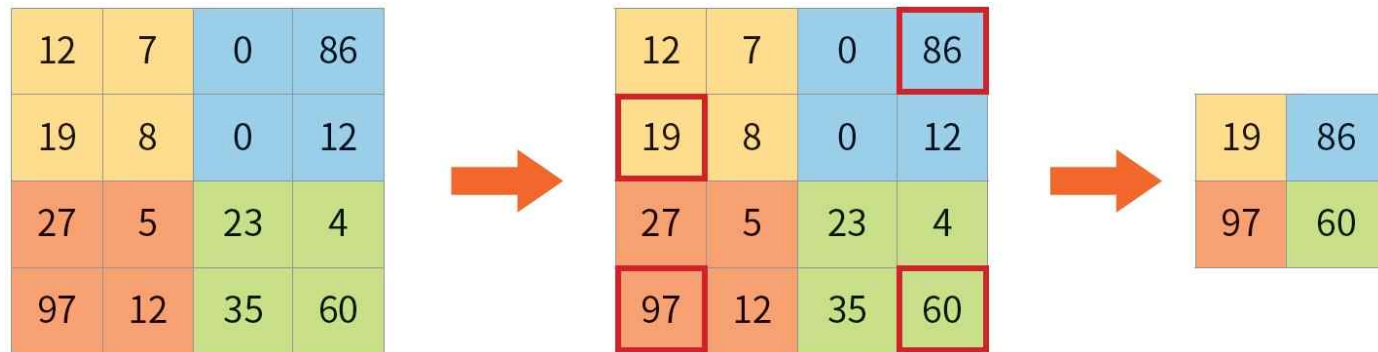
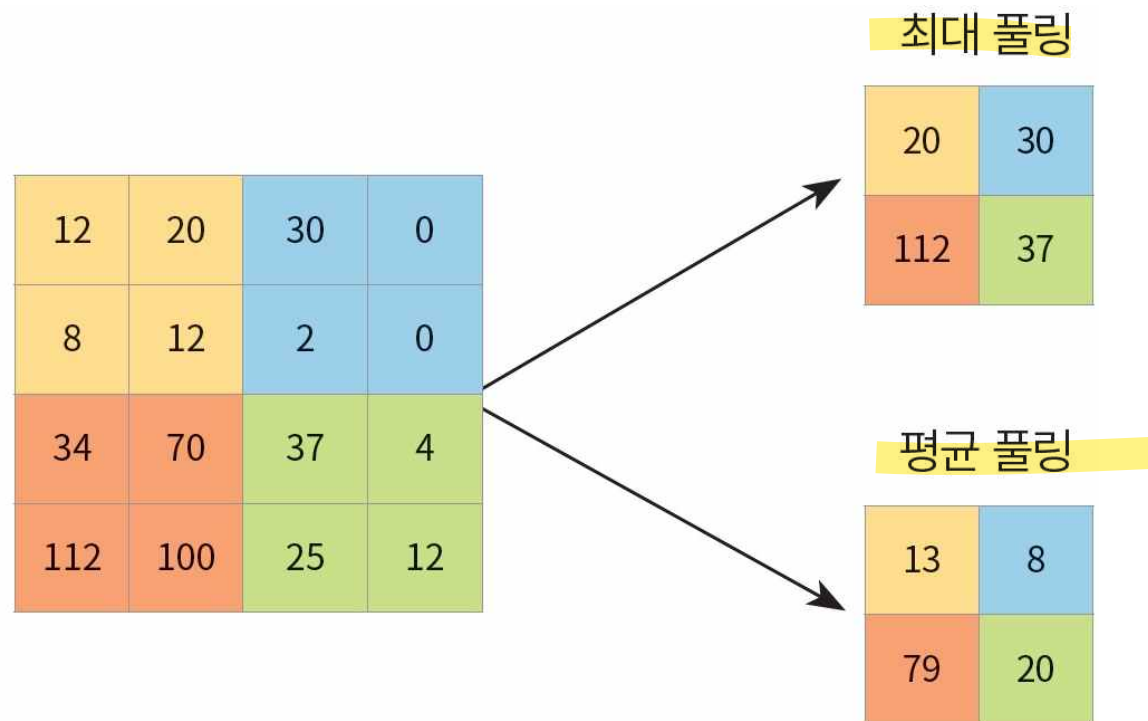


그림 15-20 풀링 연산

풀링의 종류



풀링의 장점

- 레이어의 크기가 작아지므로 계산이 빨라진다.
- 레이어의 크기가 작아진다는 것은 신경망의 매개변수가 작아진다는 것을 의미한다. 따라서 과적합이 나올 가능성이 줄어든다.
- 공간에서 물체의 이동이 있어도 결과는 변하지 않는다. 즉 물체의 공간이동에 대하여 둔감해지게 된다.
 - 평행이동-불변 (translation-invariant)

풀링의 장점

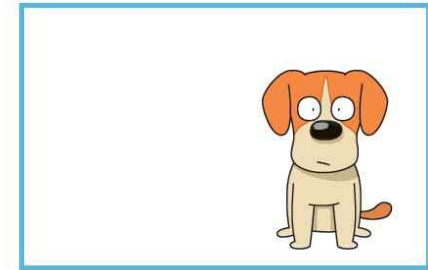
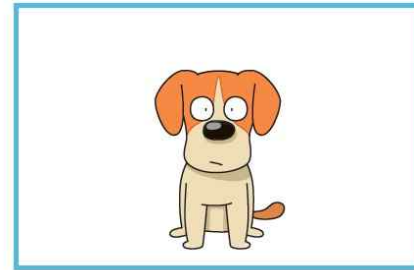
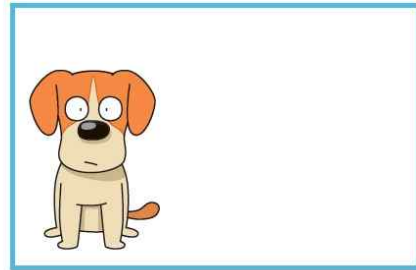
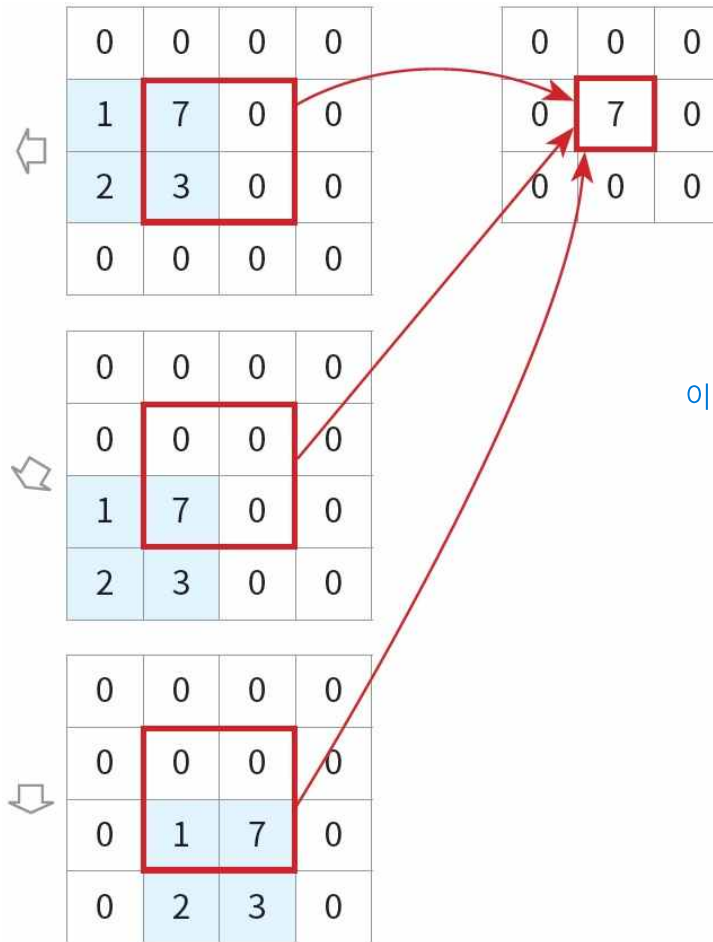


그림 15-22 평행이동된 영상

풀링의 장점

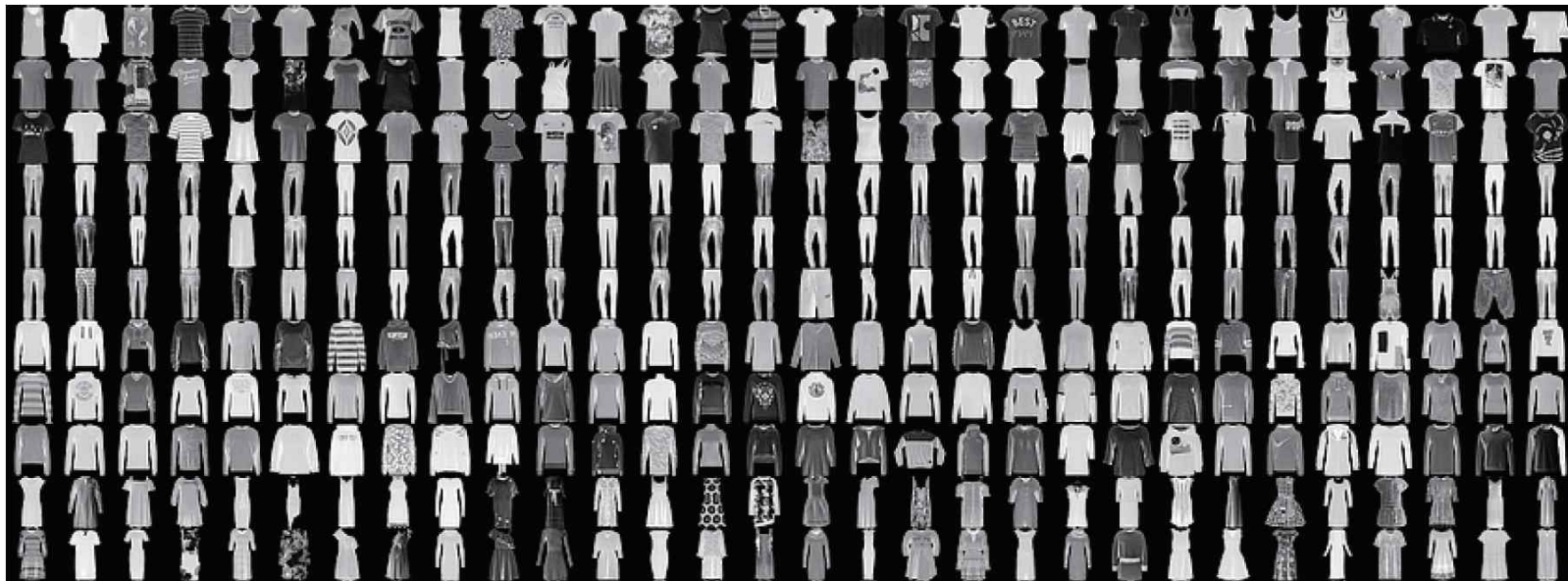


Q

- 풀링(Pooling)이란 어떤 연산인가?

DNN을 이용한 영상 분류

- 텐서플로우 튜토리얼에 나오는 패션 아이템을 DNN으로 분류하는 코드
 - 10개 범주(category)의 약 70,000개의 패션 관련 이미지, 각 크기 28*28
 - 70,000개 중에서 60,000개는 훈련 데이터, 10,000개는 테스트 데이터로



DNN을 이용한 영상 분류

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import datasets, layers, models

fashion_mnist = keras.datasets.fashion_mnist
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()

train_images = train_images / 255.0 #broadcast
test_images = test_images / 255.0

model = models.Sequential()
model.add(layers.Flatten(input_shape=(28, 28)))
model.add(layers.Dense(128, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))
```

DNN을 이용한 영상 분류

```
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])  
  
model.fit(train_images, train_labels, epochs=5)
```

실행결과

```
...
Epoch 1/5
60000/60000 [=====] - 4s 70us/sample - loss: 0.4988 -
accuracy: 0.8264
Epoch 2/5
60000/60000 [=====] - 4s 68us/sample - loss: 0.3792 -
accuracy: 0.8629
Epoch 3/5
60000/60000 [=====] - 4s 66us/sample - loss: 0.3394 -
accuracy: 0.8766
Epoch 4/5
60000/60000 [=====] - 4s 66us/sample - loss: 0.3142 -
accuracy: 0.8859
Epoch 5/5
60000/60000 [=====] - 4s 65us/sample - loss: 0.2977 -
accuracy: 0.8907
```

CNN을 이용한 영상 분류

- 이번에는 동일한 문제를 컨벌루션 신경망을 이용하여서 수행해보자. 우리는 **Conv2D**와 **MaxPooling2D** 층을 쌓아서 컨벌루션 신경망을 구축할 것이다

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import datasets, layers, models

fashion_mnist = keras.datasets.fashion_mnist
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()
```

CNN을 이용한 영상 분류

```
train_images = train_images.reshape((60000, 28, 28, 1)) # conv2d 입력은 3차원
test_images = test_images.reshape((10000, 28, 28, 1)) # 28*28 -> 28*28*1
# 행*열*채널

train_images = train_images / 255.0 # broadcast
test_images = test_images / 255.0

model = models.Sequential()

model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(layers.MaxPooling2D((2, 2))) # 크기가 준다
model.add(layers.Conv2D(64, (3, 3), activation='relu')) # 3,3 마스크를 64개 짜리에 적
model.add(layers.MaxPooling2D((2, 2))) # 그 결과에 대해서 크기를 줄인
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
```

필터의 차원

CNN을 이용한 영상 분류

```
model.add(layers.Flatten())      짝 퍼버린다
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(train_images, train_labels, epochs=5)

test_loss, test_acc = model.evaluate(test_images, test_labels)
```

중간결과

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_2 (MaxPooling2)	(None, 13, 13, 32)	0
conv2d_4 (Conv2D)	(None, 11, 11, 64)	18496
max_pooling2d_3 (MaxPooling2)	(None, 5, 5, 64)	0
conv2d_5 (Conv2D)	(None, 3, 3, 64)	36928
flatten_2 (Flatten)	(None, 576)	0
dense_4 (Dense)	(None, 64)	36928
dense_5 (Dense)	(None, 10)	650
Total params: 93,322		
Trainable params: 93,322		
Non-trainable params: 0		

실행결과

```
poch 1/5
60000/60000 [=====] - 24s 396us/sample - loss: 0.4912 -
acc: 0.8198
Epoch 2/5
60000/60000 [=====] - 24s 394us/sample - loss: 0.3232 -
acc: 0.8832
Epoch 3/5
60000/60000 [=====] - 24s 392us/sample - loss: 0.2744 -
acc: 0.8995
Epoch 4/5
60000/60000 [=====] - 23s 391us/sample - loss: 0.2435 -
acc: 0.9099
Epoch 5/5
60000/60000 [=====] - 23s 392us/sample - loss: 0.2187 -
acc: 0.9186
10000/10000 [=====] - 2s 151us/sample - loss: 0.2693 -
acc: 0.9049
정확도: 0.9049
```

Q

- 컨볼루션 신경망은 일반적인 심층 신경망에 비하여 어떤 장점을 가지는가?

왜 성능이 더 좋아 졌을까요? 완전연결 쓰는 것 보다.

컨볼루션을 안쓰면 앞에 노드를 전부 받아드린다, 하지만 컨볼루션 마스크를 하면 조금씩만 뽑아낸다.

컨볼루션은 부분정보만 취합을 한다, 일반 적인 것은 전부 취합을 한다.

필터를 여러 개 해서 전부 다 하는 것이 아닌 조금씩 계산한다.

결과적으로 잊이 오버 피팅을 줄여준다는 장점이 있다. 한꺼번에 다 고려하는 것보다 부분 정보만 처리하는 것이 오버피팅을 그렇게 해서 성능이 향상된 것이다.

층이 높다면 높은 쪽이 계산량이 많지만 층이 똑같다면 완전 연결이 계산량이 더 많다.

Summary

- 영상 인식에 많이 사용되는 신경망은 컨벌루션 인공신경망(**CNN**)이다. 컨벌루션 신경망(**CNN**)은 동물의 조직에서 영감을 얻어서 만들어진 신경망이다.
- **컨벌루션(Convolution Neural Network: CNN)** 신경망에서는 하위 레이어의 노드들과 상위 레이어의 노드들이 부분적으로만 연결되어 있다. 따라서 복잡도가 낮아지고 과대 적합에 빠지지 않는다.
- 컨벌루션은 주변 화소값들에 가중치를 곱해서 더한 후에 이것을 새로운 화소값으로 하는 연산이다.
- **풀링(Pooling)**이란 서브 샘플링이라고도 하는 것으로 입력 데이터의 크기를 줄이는 것이다.

Q & A

